Complete Technical Project Report

A Machine Learning application that classifies emails as Spam or Not Spam
using Natural Language Processing and Naive Bayes Classification

Prepared for: Project Presentation | 2024

Table of Contents

1. Project Overview

What Is This Project?

Spam Filter AI is a desktop application that uses Machine Learning (ML) to automatically decide whether an email is spam (unwanted junk mail) or ham (a legitimate email). You paste an email into a window, press a button, and within milliseconds the application tells you the verdict.

The Problem It Solves

Every day, roughly 45% of all emails sent worldwide are spam. Manually reading each email to check is impossible. A spam filter automates this by learning patterns from thousands of labelled emails (ones humans have already marked as spam or not), then applying those learned patterns to new, unseen emails.

How It Works — In Plain English

The application works in three phases:

Phase 1 — Training (done once, in Google Colab): The model reads 5,000+ already-labelled emails, learns which words are common in spam vs legitimate email, and saves this knowledge to a file.

Phase 2 — Saving: The trained knowledge is saved as two .pkl files (spam_detector_model.pkl and tfidf_vectorizer.pkl) that can be loaded instantly.

Phase 3 — Prediction (the GUI): When you paste a new email, the app cleans the text, converts it to numbers using the saved vectorizer, and feeds it to the saved model which outputs: SPAM or NOT SPAM.

Key Achievements

95.2%	98%	< 1s
Overall Accuracy	*Ham Precision (won't delete real emails)*	*Prediction Time*

2. Technologies & Libraries Used

Every piece of technology used in this project is explained below — what it is, what it does in this project, and why it was chosen over alternatives.

Technology	Role in This Project	Why This, Not Something Else?
Python 3.12	Main programming language for all files	Dominates data science/ML. Vast library support. Easy syntax.
pandas	Loads the CSV dataset into a table-like structure called a DataFrame	Industry standard for tabular data. Much easier than plain Python file reading.
NumPy	Numerical array operations used internally by scikit-learn	The backbone of all scientific Python. Required by almost every ML library.
scikit-learn	Provides TF-IDF vectorizer, Naive Bayes model, train/test split, and metrics	The most popular general-purpose ML library. Well-documented, battle-tested.
NLTK	Provides the English stopwords list and the Snowball stemmer	Longest-standing NLP library in Python. Perfect for text preprocessing tasks.
joblib	Saves and loads the trained model and vectorizer as .pkl files	Part of scikit-learn ecosystem. Faster than pickle for large numpy arrays.
tkinter	Creates the desktop Graphical User Interface (GUI) window	Comes built into Python. No installation needed. Sufficient for simple GUIs.
Google Colab	Cloud environment where training was run	Free GPU/CPU access. No local setup needed. Ideal for quick ML experiments.
Kaggle	Source of the labelled spam email dataset	Largest open dataset repository. Free, high-quality, labelled datasets.

3. Project File Structure

The project is split into multiple files, each with a single responsibility. This is called Separation of Concerns — a core software engineering principle.

File	What It Does	When It Runs
`data_preprocessing.py`	Cleans raw email text: lowercasing, removing punctuation/HTML/numbers, stripping stopwords, stemming words	Once, before training. Produces preprocessed_emails.csv
`feature_extraction.py`	Converts cleaned text into a numerical matrix using TF-IDF. Each email becomes a row of numbers the model can understand.	Once, after preprocessing. Produces X_features.pkl and tfidf_vectorizer.pkl
`model.py`	Splits data into train/test sets, trains the Naive Bayes classifier, saves the trained model	Once, after feature extraction. Produces spam_detector_model.pkl
`evaluation.py`	Same as model.py but also saves X_test.pkl and y_test.pkl for later evaluation. Measures accuracy, precision, recall, F1.	Once, after training. Used to verify model performance.
`gui.py`	Creates the visual desktop window. Loads the saved model and vectorizer. Takes user input, preprocesses it, and shows prediction.	Every time the user wants to check an email
`__init__.py`	Empty file that tells Python to treat the src/ folder as a package, enabling imports like 'from src.data_preprocessing import ...'	Automatically, by Python's import system
`spam_detector_model.pkl`	The trained model saved to disk. Contains all the learned probability weights from training.	Loaded by gui.py at startup
`tfidf_vectorizer.pkl`	The fitted TF-IDF vectorizer saved to disk. Must use the same vectorizer that was used during training.	Loaded by gui.py at startup

4. The Machine Learning Pipeline

A Machine Learning pipeline is the sequence of steps that transforms raw data into a working prediction system. Think of it like a factory assembly line — each station performs one job and passes its output to the next.

Step	Action	Input	Output
1	Load raw data	emails.csv (5,172 raw emails)	Pandas DataFrame
2	Text Preprocessing	Raw email text with HTML, numbers, punctuation	Clean stemmed tokens
3	Feature Extraction (TF-IDF)	Cleaned text strings	5172 x 3000 numerical matrix
4	Train/Test Split	Full feature matrix	80% train, 20% test
5	Model Training	Training features + labels	Trained Naive Bayes model
6	Model Evaluation	Test features + true labels	Accuracy 95.2%, F1 scores
7	Serialisation	Trained model + vectorizer objects	.pkl files saved to disk
8	Prediction (GUI)	New email pasted by user	SPAM or NOT SPAM verdict

Note: Steps 1-7 are the training pipeline (run once). Step 8 is the inference pipeline (run every time a user submits an email).

5. Data Preprocessing (data_preprocessing.py)

Why Preprocess At All?

Raw email text is messy. It contains HTML tags, random capitalisation, numbers, punctuation and filler words like 'the', 'is', 'a'. These add noise without helping the model distinguish spam from ham. Preprocessing standardises and cleans the text so the model can focus on meaningful signal.

Each Preprocessing Step Explained

Step 1 — Lowercasing

```
text = text.lower()
```

'FREE OFFER' and 'free offer' are the same thing. Converting everything to lowercase ensures the model treats them identically instead of as two different words.

Step 2 — Remove HTML Tags

```
text = re.sub(r\'<[^>]+>\', \' \', text)
```

Spam emails often contain HTML code like <b>, <font>, <a href=...>. These tags carry no semantic meaning about spam vs ham, so they are stripped out using a Regular Expression (regex) pattern.

Step 3 — Remove Numbers

```
text = re.sub(r'\d+', ' ', text)
```

Numbers like phone numbers, prices (e.g. '1000000') are often in spam but vary too much to be useful features. Removing them reduces noise.

Step 4 — Remove Punctuation

```
text = re.sub(r'[^\w\s]', ' ', text)
```

Exclamation marks, commas, brackets etc. are removed. The regex [^\w\s] means 'remove anything that is not a word character or whitespace'.

Step 5 — Remove Extra Whitespace

```
text = re.sub(r'\s+', ' ', text).strip()
```

After all the removals above, the text has many extra spaces. This step collapses them all into single spaces and removes leading/trailing whitespace.

Step 6 — Remove Stopwords

```
tokens = [w for w in tokens if w not in stop_words]
```

Stopwords are common English words like 'the', 'is', 'at', 'which', 'on'. They appear in both spam and ham equally, so they don't help the model decide. NLTK provides a list of 179 English stopwords that are filtered out.

Step 7 — Stemming

```
tokens = [stemmer.stem(word) for word in tokens]
```

Stemming reduces words to their root form. 'running', 'runs', 'ran' all become 'run'. 'winning', 'winner', 'wins' all become 'win'. This means the model doesn't treat these as separate features — they all count as the same word. The Snowball Stemmer is used because it's more accurate than the older Porter Stemmer.

Before vs After Preprocessing — Example

Before Preprocessing	After Preprocessing
`"CONGRATULATIONS! You have WON $1,000,000!! **Click HERE** to claim your prize now!!!"`	`"congratul won click claim prize"`
`"Hi John, please find attached the Q3 report for your review."`	`"hi john pleas find attach q report review"`

6. Feature Extraction (feature_extraction.py)

The Core Problem: Computers Don't Understand Words

Machine learning models work with numbers — not text. We need a way to convert each email (a string of words) into a row of numbers that captures the meaning and importance of the words. This is called Feature Extraction.

What Is TF-IDF?

TF-IDF stands for Term Frequency — Inverse Document Frequency. It is a numerical statistic that reflects how important a word is to a document relative to a collection of documents. It has two components:

TF (Term Frequency)

How often a word appears in a specific email. If 'free' appears 5 times in a 100-word email, TF = 5/100 = 0.05. The more a word appears, the higher its TF score.

IDF (Inverse Document Frequency)

How rare or common a word is across ALL emails. If 'free' appears in 1,000 out of 5,000 emails, it gets a lower IDF score (it's common, so less informative). If 'viagra' appears in only 50 emails, it gets a high IDF score (rare, very informative). IDF = log(Total Emails / Emails containing the word).

TF-IDF Score = TF x IDF

A word that appears frequently in one email but rarely across the whole dataset gets the highest TF-IDF score — it is highly distinctive and informative. Common words like 'the' appear everywhere, so their IDF is near zero, making their TF-IDF score near zero even if they appear many times.

The Resulting Matrix

After TF-IDF transformation, you have a matrix with one row per email and one column per unique word (feature). With max_features=3000, this means:

Rows	5,172	One row per email in the dataset
Columns	3,000	The 3,000 most important words across all emails
Each cell value	0.0 to 1.0	TF-IDF score of that word in that email (0 = word absent)
Matrix density	Very sparse	Most cells are 0 because most emails don't contain most words

Why TF-IDF and Not Simple Word Counting?

Simple word counting (called Bag of Words or CountVectorizer) doesn't account for word rarity. The word 'the' would score very high for every email, adding no value. TF-IDF penalises common words automatically, making the model focus on discriminating words like 'viagra', 'lottery', 'winner', which are strong spam signals.

7. The Naive Bayes Algorithm — Deep Dive

The Core Idea

Naive Bayes is a probabilistic classifier based on Bayes' Theorem from probability theory. It answers the question: Given that this email contains these words, what is the probability that it is spam?

Bayes' Theorem — The Foundation

The theorem states that the probability of a hypothesis (H) given evidence (E) is:

```
P(Spam | words) = [ P(words | Spam) x P(Spam) ] / P(words)
```

In plain English:

The probability this email is spam, given the words it contains, equals: how likely these words are to appear in spam emails, multiplied by the overall probability of any email being spam, divided by the overall probability of seeing these words.

Why 'Naive'?

The 'naive' assumption is that all words are independent of each other. In reality, words like 'click' and 'here' often appear together in spam. The model pretends they are unrelated — this is the naive part. Despite this unrealistic assumption, Naive Bayes performs surprisingly well on text classification in practice.

Why Multinomial Naive Bayes Specifically?

There are three variants of Naive Bayes:

Variant	Best For	Used Here?
Multinomial NB	Word counts and TF-IDF scores (integers or frequencies). Perfect for text.	YES — our TF-IDF values are frequency-like scores
Bernoulli NB	Binary features (word present: yes/no). Good for short texts.	No — we have continuous TF-IDF values, not binary
Gaussian NB	Continuous numeric features that follow a normal distribution.	No — text data is not normally distributed

How The Training Works — Step By Step

1. **Count class frequencies**: The model counts: how many training emails are spam? How many are ham? In the dataset: roughly 1,500 spam and 3,600 ham. This gives P(Spam) = 0.29, P(Ham) = 0.71.
2. **Build word probability tables**: For every word in the vocabulary, the model counts: how often does this word appear in spam emails? How often in ham? Example: 'free' might appear 800 times in spam emails but only 20 times in ham.
3. **Apply Laplace smoothing**: To avoid zero probabilities (a word never seen in training spam doesn't mean it can't appear), a small count of 1 is added to every word count. This is called Laplace smoothing or add-one smoothing.
4. **Store log probabilities**: Since multiplying thousands of tiny probabilities together causes numerical underflow (the number becomes too small for the computer), all probabilities are converted to logarithms and added instead of multiplied.

How Prediction Works — A Concrete Example

Suppose a new email contains just three words after preprocessing: "free winner click"

The model calculates two scores:

```
Score(Spam) = log P(Spam) + log P('free'|Spam) + log P('winner'|Spam) + log P('click'|Spam)
```

```
Score(Ham) = log P(Ham) + log P('free'|Ham) + log P('winner'|Ham) + log P('click'|Ham)
```

If Score(Spam) > Score(Ham), the email is classified as SPAM. Otherwise, NOT SPAM.

8. Why Naive Bayes? — Alternatives Compared

This project could have used many different machine learning algorithms. Here is a detailed comparison of the main alternatives:

Algorithm	Accuracy on Text	Training Speed	Interpretability	Verdict
Multinomial Naive Bayes	High (95%+)	Extremely fast (< 1 second)	High — probabilities are human-readable	CHOSEN — best speed/accuracy tradeoff for this task
Logistic Regression	High (96%+)	Fast (seconds)	Medium — weights show word importance	Good alternative but marginally slower to train
Random Forest	Medium-High (93%)	Moderate (minutes)	Low — black box ensemble	Overkill for text. TF-IDF + RF is less natural than NB
SVM (Support Vector Machine)	Very High (97%+)	Slow on large data	Low — kernel trick is hard to explain	Better accuracy but much harder to explain and tune
BERT / Transformers	State of the art (99%+)	Very slow (hours)	Very Low — billions of parameters	Massive overkill. Requires GPU, huge memory, complex setup
Deep Learning (LSTM/RNN)	High (97%+)	Slow (30+ mins)	Very Low — neural network	Requires large datasets and GPU. Not justified for 5K emails

The green row is the chosen algorithm. For a dataset of ~5,000 emails with a simple binary classification task, Naive Bayes is the textbook-optimal choice.

9. Model Training & Saving (model.py)

Train/Test Split

Before training, the dataset is split into two parts using train_test_split with test_size=0.2 and random_state=42:

Training Set (80%)	80%	~4,137 emails	Model learns from this data
Test Set (20%)	20%	~1,035 emails	Evaluates model on unseen data

Why 80/20? It's the industry standard. Enough data to train effectively, and enough held-out data to get a reliable estimate of real-world performance. random_state=42 ensures the split is reproducible — you get the same split every time.

Model Serialisation with joblib

After training, the model is saved to disk using joblib.dump(). This is called serialisation — converting a Python object in memory into a binary file on disk. The .pkl extension stands for pickle, Python's native object serialisation format. joblib is preferred over pickle for scikit-learn models because it handles large NumPy arrays more efficiently.

Why Save the Model?

Training happens once. If the model wasn't saved, every time you opened the GUI you'd have to retrain from scratch, taking seconds each time. With saving, the GUI loads the pre-trained model in milliseconds. This is the standard production pattern for any ML application.

Why Save the Vectorizer Too?

This is critical and often misunderstood. The TF-IDF vectorizer must be the same object used during training. When you call vectorizer.fit_transform() during training, it builds a specific vocabulary (e.g., column 47 = 'free', column 892 = 'winner'). If you created a new vectorizer at prediction time, column 47 might mean something completely different, making the prediction meaningless. The saved vectorizer guarantees consistency.

10. Model Evaluation (evaluation.py)

The Four Key Metrics Explained

Accuracy alone is not enough to evaluate a classifier. A model that labels every email as 'not spam' would be 71% accurate on this dataset — but completely useless. We use four metrics:

Metric	Formula	Plain English	Why It Matters
Accuracy	Correct predictions / Total predictions	Out of all emails, how many did we classify correctly?	Overall health check of the model
Precision	True Positives / (True Positives + False Positives)	Of all emails we called SPAM, how many actually were spam?	Low precision = real emails wrongly deleted
Recall	True Positives / (True Positives + False Negatives)	Of all actual spam emails, how many did we catch?	Low recall = spam getting through to inbox
F1 Score	2 x (Precision x Recall) / (Precision + Recall)	The harmonic mean of precision and recall. Balances both.	Single number summarising classifier quality

The Confusion Matrix Concept

Every prediction falls into one of four buckets:

	Predicted: Ham	Predicted: Spam
Actual: Ham	TRUE NEGATIVE (TN) Correctly identified as not spam	FALSE POSITIVE (FP) Real email wrongly marked as spam!
Actual: Spam	FALSE NEGATIVE (FN) Spam email missed, reaches inbox	TRUE POSITIVE (TP) Spam correctly caught

11. Your Model's Results — Interpretation

Your Actual Results

Here is the classification report your model produced, with each number explained:

Class	Precision	Recall	F1-Score	Support	What This Means
Ham (0)	0.98	0.95	0.97	742	98% of emails labelled 'not spam' are actually not spam. Very safe.
Spam (1)	0.89	0.95	0.92	293	89% of spam calls are correct. Catches 95% of all actual spam.
Overall	0.95	0.95	0.95	1035	95.2% of all emails classified correctly.

Why 95% Is an Excellent Result

For a spam filter built with a simple model, a small dataset, and no hyperparameter tuning, 95.2% accuracy is outstanding. The 98% ham precision is the most critical number — it means the filter almost never deletes a real, important email, which would be the most damaging kind of error. The model correctly catches 95% of all spam.

Why Training Took Less Than 1 Second

Naive Bayes training is a single pass through the data — it counts frequencies and computes log probabilities. There is no iterative optimization, no gradient descent, no back-propagation. On a dataset of ~5,000 emails with 3,000 features, even a basic CPU can do this in milliseconds. This is one of Naive Bayes's biggest advantages.

12. The Graphical User Interface (gui.py)

What Is tkinter?

tkinter is Python's built-in GUI library. It creates native desktop windows using the operating system's own widget toolkit. It comes pre-installed with Python — no additional installation is needed. It's the simplest way to create a GUI in Python for a project of this scale.

GUI Components Explained

Widget	tkinter Class	Purpose
Main window	`tk.Tk()`	The root application window. 600x500 pixels. Light grey background.
Header banner	`tk.Frame + tk.Label`	Blue banner at the top showing 'Spam Filter AI' title
Text area	`tk.Text`	Multi-line input where the user pastes email content
Submit button	`tk.Button`	Triggers process_email() method when clicked
Delete button	`tk.Button`	Clears the text area and resets the status label
Status label	`tk.Label`	Shows the result: 'SPAM' or 'NOT SPAM'
Footer	`tk.Frame + tk.Label`	Blue banner at bottom with contact info

The process_email() Method — What Happens When You Click Submit

1. Get text from the tk.Text widget using email_text.get('1.0', tk.END)
2. Pass the raw text through preprocess_text() — the same function used during training
3. Transform the cleaned text using vectorizer.transform() — converts it to a TF-IDF vector
4. Feed the vector to model.predict() — returns 0 (ham) or 1 (spam)
5. Update the status_label text to show the result to the user

Important: The GUI uses vectorizer.transform(), not fit_transform(). transform() applies the existing vocabulary learned during training. fit_transform() would learn a new vocabulary from the single email, which would be completely wrong.

13. The Dataset

Dataset Overview

Source	Kaggle — spam-mails-dataset by venky73
Format	CSV (Comma-Separated Values)
Total emails	~5,172
Ham emails (label_num = 0)	~3,672 (71%)
Spam emails (label_num = 1)	~1,500 (29%)
Text column	'text' — preprocessed email body
Label column used	'label_num' — 0 for ham, 1 for spam

Class Imbalance

The dataset is imbalanced — 71% ham vs 29% spam. This is realistic (most real emails are not spam) but means a naive model that always predicts 'ham' would already be 71% accurate. This is why we look at precision and recall per class, not just overall accuracy. Our model achieves 95% recall on spam despite the imbalance, which means it genuinely learned the distinction and is not just guessing 'ham' most of the time.

Why This Dataset?

This dataset is derived from the famous Enron email corpus — a real-world collection of emails from Enron Corporation employees, augmented with known spam messages. It is widely used in academic spam classification research, making it a credible and well-validated benchmark for this task.

14. Complete Workflow Summary

Here is the entire project from start to finish on one page:

Phase	Where	Command / Action	What Happens
1. Get data	Kaggle	Download emails.csv	5,172 labelled emails downloaded
2. Upload	Google Colab	files.upload()	CSV and .py files uploaded to Colab environment
3. Preprocess	Colab Cell 3	preprocess_text() on df['text']	Text cleaned, stemmed, stopwords removed
4. Extract features	Colab Cell 4	TfidfVectorizer.fit_transform()	5172x3000 numerical matrix created
5. Train	Colab Cell 5	MultinomialNB().fit(X_train, y_train)	Model learns spam/ham word probabilities
6. Evaluate	Colab Cell 6	accuracy_score(), classification_report()	95.2% accuracy confirmed
7. Download	Colab → Linux	files.download() + mv command	.pkl files moved to project root
8. Run GUI	Linux Terminal	python3 src/gui.py	App window opens, ready for email input
9. Use	GUI Window	Paste email → Submit Email	Model returns SPAM or NOT SPAM in < 1 second

15. Glossary — Plain English Definitions

Algorithm — A set of rules or instructions a computer follows to solve a problem. Like a recipe.

Bag of Words — A simple text representation that counts how many times each word appears, ignoring word order.

Classification — A type of ML task where the output is a category (e.g., spam/ham, yes/no, cat/dog).

CSV — Comma-Separated Values. A spreadsheet-like text file where each row is a record and columns are separated by commas.

DataFrame — A pandas data structure like a spreadsheet table — rows and named columns.

Feature — An individual measurable property used as input to a model. In this project, each word's TF-IDF score is a feature.

Feature Extraction — The process of converting raw data (text) into numerical features a model can process.

Ham — Legitimate, non-spam email. The term comes from amateur radio slang.

Hyperparameter — A setting that controls the learning process itself (not learned from data). E.g., max_features=3000.

IDF — Inverse Document Frequency. Measures how rare a word is across all documents. Rare words get higher scores.

Joblib — A Python library for efficiently saving and loading Python objects (especially large numerical arrays) to/from disk.

Label — The known correct answer for a training example. In this dataset, 0 = ham, 1 = spam.

Laplace Smoothing — Adding a small count to prevent zero probabilities when a word was never seen in training.

Log Probability — The logarithm of a probability. Used to prevent numerical underflow when multiplying many tiny numbers.

Model — A mathematical function that maps inputs (features) to outputs (predictions), learned from training data.

Naive Bayes — A probabilistic classifier based on Bayes' Theorem that assumes feature independence.

NLTK — Natural Language Toolkit. A Python library for text processing tasks like tokenisation, stemming, and stopword removal.

NLP — Natural Language Processing. The field of AI that deals with understanding and processing human language.

Overfitting — When a model learns the training data too well and fails to generalise to new data.

Pickle (.pkl) — Python's format for serialising (saving) objects to disk so they can be loaded later.

Pipeline — A sequence of data processing steps where the output of each step becomes the input of the next.

Precision — Of all items classified as positive, what fraction are truly positive? High precision = few false alarms.

Recall — Of all truly positive items, what fraction did the model correctly identify? High recall = few misses.

Regex — Regular Expression. A pattern-matching language for text. Used to remove HTML tags, numbers, punctuation.

Serialisation — Converting an in-memory object into a format that can be stored on disk or transmitted.

Spam — Unsolicited bulk email, typically commercial or malicious in nature.

Stemming — Reducing words to their root form. 'Running' becomes 'run'. Reduces vocabulary size.

Stopwords — Common words (the, is, at) that appear in almost all text and carry little discriminating information.

TF — Term Frequency. How often a word appears in a specific document relative to the document's total words.

TF-IDF — Term Frequency-Inverse Document Frequency. A numerical measure of word importance in a document.

tkinter — Python's standard built-in library for creating desktop GUI applications.

Token — A single word or word-like unit after text has been split (tokenised).

Train/Test Split — Dividing the dataset into a training portion (model learns) and test portion (model is evaluated).

Vectorizer — An object that transforms text into a numerical vector. In this project: TfidfVectorizer.

Built with Python · scikit-learn · NLTK · tkinter · Google Colab

95.2% Accuracy	**5,172 Emails Trained**	**3,000 Features**	**< 1s Prediction**